

Coding JS In MongoDB

Database And Collection Objects

- MongoClient is the first object you must create in your code. This represents your connection to the MongoDB server. This is equivalent to the Mongo Shell. We pass the DB URI and connection parameters and it creates a connection to the database.
- The **database** object represents the database and allows us to access its collections.
- The **collection** object represents a particular collection and it is used to run queries against that collection.
- A MongoClient can create multiple **database** objects and a **database** object can create multiple **collection** objects.
- There are 2 connection parameters to be passed to the MongoClient: the **Mongo Server URL** and any **additional options**.
- It is also possible to specify the **options** in the server URL itself.
- **Exercise 8.01, Page - 180**

Creating and Executing Find Queries

- Before we can execute find queries, we need to connect to the database using MongoClient.

```
const MongoClient = require('mongodb').MongoClient;

// Replace this variable with the connection string for your server,
// provided by
// MongoDB Atlas.
const url = 'mongodb+srv://username:password@server-abcdef.gcp.mongodb.net/test';
```

```
const client = new MongoClient(url);

client.connect(function(err) {
  console.log('Connected to MongoDB with NodeJS!');
  // OUR CODE GOES BELOW HERE

  // AND ABOVE HERE
  client.close();
})
```

- After doing that, we create a database object.

```
// OUR CODE GOES BELOW HERE
const database = client.db("sample_mflix")
// AND ABOVE HERE
```

- When sending queries in the mongo shell, you must pass a document to the command as a filter for your documents.

```
// Defining filter first.
var filter = { title: 'Blacksmith Scene'};
database.collection("movies").find(filter).toArray(function(err, docs) {
});

// Doing everything in a single line.
database.collection("movies").find({title: 'Blacksmith
Scene'}).toArray(function(err, docs) {});
```

- The **toArray** at the end of the statement is added as the **find()** command will return a **cursor**.

Using Cursors and Query Results

- **toArray**: This will take all the results of the query and place them in a single array. This is easy to use but not very efficient when you are expecting a large result from your query.
- **each**: This will iterate through each document in the result set, one at a time. This is a good pattern if you want to inspect or use each document in the result.
- **next**: This will allow you to access the next document in the result set. This is the best pattern to use if you are only looking for a single document or a subset of your results without having to iterate through the entire result.

You can also accomplish the same **sort** and **limit** functionality from the mongo shell by placing these commands after **find**.

Exercise 8.02 - Page 387

Callbacks in NodeJS

- A callback is a function provided as a parameter to a second function, which allows both functions to be run in order.
- Without using callbacks (or any other synchronization pattern), both functions would start executing right after the other. When using a driver, this would create errors, because the second function may be dependent on the first function finishing before it begins.
- The parameters passed into our callback function are determined by the first function that we provide our callback function to. When **passing a function, fA, as a parameter to a second function, fB, the parameters of fA are provided by fB**.
- So, our callback function, **function(err, doc) { ... }**, is provided as a parameter to the driver function, **each**.

```

database.collection(collectionName).find(query).limit(2).
project({title: 1}).each
(function(err, doc) {
    if(doc) {
        console.log('Current doc');
        console.log(doc);
    } else {
        client.close(); // Close our connection.
        return false;    // End the each loop.
    }
});

```

The same example but with **console.log** statements.

```

console.log('This will execute first.')
database.collection(collectionName).find(query).limit(2).
project({title: 1}).each
(function(err, doc) {
    console.log('This will execute last, once for each document in the
    result.')
    if(doc) {
    } else {
        client.close(); // Close our connection.
        return false;    // End the each loop.
    }
});
console.log('This will execute second.');
```

It executes as follows:

```

Connected to MongoDB with NodeJS!
This will execute first.
This will execute second.
This will execute last, once for each doc.
This will execute last, once for each doc.
This will execute last, once for each doc.
```

Callbacks are sometimes complicated patterns to become familiar with and are increasingly being replaced by more advanced Node.js patterns, such as **promises** and **async/await**.

Basic Error Handling in NodeJS

- In the MongoDB driver, most commands that can return an error in the mongo shell can also return an error in the driver.
- In the MongoDB driver, most commands that can return an error in the mongo shell can also return an error in the driver. In the case of callbacks, the **err parameter** will always exist; however, **if there is no error, the value of err is null**.
- This **"error-first"** pattern to catch errors in asynchronous code is standard practice in NodeJS.

Note: It is advisable to use callbacks to check the parameters that are passed in before we try to use them. In the case of a find command, this would mean checking whether there is an error and checking that a document was returned. When writing code against MongoDB, it is good practice to validate everything that was returned from the database and log errors for debugging purposes.

Exercise 8.03 - Error Handling and Callbacks with the Node.js Driver - Page 395

Inserting Data with the NodeJS Driver

- Similar to the mongo shell, we can use either the **insertOne** or **insertMany** function to write data into our collection. **These functions are called on the collection object**. The only parameter we need to pass into these functions is a **single document in the case of insertOne, or an array of documents in the case of insertMany**.

```
database.collection(collectionName).insertOne({Hello:
  "World"}, function(err, result) {
  // Handle result.
})

database.collection(collectionName).insertMany([{Hello: "World"},
  {Hello: "Mongo"}], function(err, result) {
  // Handle result.
})
```

- We pass a callback to these functions to handle the result of the operation. Insert operations will return **an error (which may be null) and a result**, which details how the insert operation executed.

Updating and Deleting Data with the NodeJS Driver

- Updating and deleting documents with the driver follows the same pattern as the insert function, where the collection object passes through a callback, checks for errors, and analyzes the results of the operation.
- All these functions will return a results document.

```
database.collection(collectionName).
updateOne({Hello: "World"}, {$set: {Hello
    : "Earth"}}, function(err, result) {
    console.log(result.modifiedCount);
    client.close();
})
```

Exercise 8.04 - Updating Data With The NodeJS Driver - Page 403

Reading Input from the Command Line

- Node.js provides a module called readline that will allow us to ask the user for input, accept that input, and then use it.
- It is best used with the **question** function that allows us to ask users questions on the command line.

```
interface.question('Hello, what is your name? ', (input) => {
    console.log(`Hello, ${input}`);

    interface.close();
});
```

Exercise 8.05: Handling Inputs in Node.js - Page 400

Activity 8.01: Creating a Simple Node.js Application Page 413